

MCP_ARCHITECTURE_PLAIN

Helios MCP Architecture — In Plain English

Plain-language companion to `docs/architecture/MCP_ARCHITECTURE.md`. For the exact spec, read that / its PDF.

In one sentence

MCP servers are vending machines of safe, pre-approved actions the AI is allowed to call — and Helios uses five of them so the AI can never write its own SQL, never do its own math, and never touch the database directly.

Why this matters to you

This is the layer that makes Helios *trustworthy* instead of just *clever*. A normal "ask your data" bot hands the model a database and a Python shell and hopes it behaves. Helios does the opposite: the model gets a fixed menu of tools and nothing else. If you build this right, "the AI hallucinated a column" or "the AI ran a \$400 query" becomes structurally impossible, not just unlikely. Get this layer wrong and every number downstream is quietly suspect.

The big ideas, simply

Grounding over generation. The model is given *tools*, *not a database handle or a Python REPL*. It picks metric names; the tools turn those into real SQL and real statistics. Three rules fall out of this:

- Only `semantic-mcp` can produce SQL (grounding).
- Only `stats-mcp` can do math (determinism).
- Only `warehouse-mcp` holds the BigQuery key (least privilege).

The five servers:

Server	Plain role	Key tools
<code>warehouse-mcp</code>	The only thing that talks to BigQuery	<code>list_tables</code> , <code>describe_table</code> , <code>dry_run</code> , <code>run_query</code> , <code>reconcile</code>
<code>semantic-mcp</code>	The only way to get SQL (builds it from approved metric definitions)	<code>get_metric</code> , <code>list_dimensions</code> , <code>build_query</code>
<code>stats-mcp</code>	The only way to do math	<code>detect_anomaly</code> , <code>decompose_change</code> , <code>significance_test</code> , <code>forecast</code> , <code>cohort_retention</code> , <code>rfm_segment</code>
<code>experiment-mcp</code>	Turns findings into properly-sized A/B tests	<code>power_analysis</code> , <code>runtime_estimate</code> , <code>design_experiment</code>
<code>report-mcp</code>	Writes the brief and remembers past runs	<code>render_brief</code> , <code>export</code> , <code>save_diagnosis</code> , <code>recall_prior</code>

The guardrail chain. Every number flows through a fixed pipeline, enforced by the tools themselves (not by asking the AI nicely): pick metric names → `semantic-mcp.build_query` makes governed SQL → `warehouse-mcp.dry_run` checks the cost *before* running → `run_query` (refuses any SQL that wasn't dry-run'd first) → `stats-mcp` does the math + `reconcile` double-checks totals → the Critic attacks it → only survivors get written into the brief.

Least privilege per agent. Each of the 7 agents sees only the tools in its allow-list (\$10 of the real doc). The Narrator literally cannot run a query; the Critic can re-query to attack a finding.

What you actually build / how it works

- **Transport:** four servers run over **stdio** (the AI spawns them as local subprocesses — fast, no network). `warehouse-mcp` runs over **HTTP** because it holds the cloud credentials and is shared across runs.
- **The two hard gates inside `warehouse-mcp`:** (1) `run_query` refuses any SQL it didn't see in a `dry_run` this run — enforced by hashing the SQL text. (2) Every query is capped at the **5 GiB** byte budget; over that, it throws `ByteBudgetExceeded` and the agent must narrow its scope.
- **The anti-hallucination core (`semantic-mcp`):** on startup it loads the registry YAML, checks every metric reference resolves (and *refuses to start* if any dangling reference exists), then builds SQL only from those approved templates. The model contributes string names — nothing else. A made-up column has no slot to live in.

- **Errors are a closed set** (the error taxonomy, §5) with codes the agent reads and self-corrects from — e.g. `ByteBudgetExceeded` → narrow and retry; `UnknownDimension` → re-plan.
- **Build order:** shared scaffolding (`base.py`) first, then `semantic-mcp` , then pair it with `warehouse-mcp` to prove the round-trip. `stats-mcp` and `experiment-mcp` are data-independent, so build them in parallel from day one.

Easy things to get wrong

- **Skipping `dry_run`** . `run_query` will reject you. Rule **G3**: every `run_query` is preceded by a `dry_run` , and any agent holding `run_query` also holds `dry_run` .
- **Letting an agent hand-write SQL "just this once."** No agent holds a raw-SQL tool. The only path to data is `build_query` → `dry_run` → `run_query` .
- **Treating the byte cap as advisory.** It's `min(your_arg, 5 GiB)` , enforced in BigQuery's job config.
- **Dumping raw rows into the model.** Results are always aggregated; row-level dumps never reach the LLM.
- **Confusing the two reconciliation backends.** Memory lives in BigQuery `helios_memory` + a vector store, *not* Postgres (the Bible's loose mention is superseded here).

Glossary — the exact words, demystified

- **MCP** — Model Context Protocol; the standard for exposing tools to an AI. Here: the vending machine.
- **`dry_run`** — ask BigQuery "how many bytes would this scan?" without running it.
- **`reconcile`** — recompute a metric's true total straight from the mart to confirm a query's answer is within 0.5% (rule **G4**).
- **governed SQL** — SQL built only from registry templates, never typed by the model.
- **`stdio` vs `HTTP`** — local subprocess pipe vs networked service.
- **byte budget** — the 5 GiB hard cap per run (`5368709120`).
- **G1–G5** — the five grounding rules (never raw SQL; never stats in prose; `dry_run` first; reconcile; canonical names only).

When to open the real doc

Open `pdf/docs/architecture/MCP_ARCHITECTURE.pdf` when you need the exact tool signatures and outputs (§6), the full error-code table (§5), the `mcp_servers.yaml` config, the reference code skeletons (§9), or the authoritative per-agent allow-list (§10).